CSE 306

Computer Architecture Sessional

Assignment 3

4-bit MIPS Design, Simulation, and Implementation

Section B2

Group 3

2005095 – Tamim Hassan Saad

2005096 – Habiba Rafique Saima

2005111 – Sadnam Faiyaz

2005114 – Shahriar Kabir

2005119 – Sadia Afrin Sithi

Introduction:

Computer processors (CPUs) are the logic circuits that respond to and process the basic instructions that drive a computer. MIPS (**Microprocessor without Interlocked Pipelined Stages**) is a RISC (**Reduced Instruction Set Computer**) ISA (**Instruction Set Architecture**). The regularity of MIPS is ensured by its set of instructions.

Here is an example:

For an Addition operation the instruction will be-

Add $t2, $t1, $t3.

This means $t2 = $t1 + $t3. Here, $t1, $t2, $t3 are registers that hold values.

A Datapath is constructed with registers, ALUs, MUXs, memories, and controls elements that can process data and addresses in the CPU. MIPS instructions are fed through a Datapath to perform various instructions like addition, load/store, branching, or jump.

The control unit sets the values for our MUX operations based on the provided instruction set for each group. To do this, we use a 16-bit ROM, where we store the hexadecimal values for the different operations.

Instruction Sets:

Our aim for this assignment is to implement a condensed and altered version of the MIPS instruction set. The term "4-bit MIPS" refers to the fact that our implementation will have a 4-bit ALU, an 8-bit address bus, and a 4-bit data bus.

As part of our design, we need to include several temporary registers, including $zero, $t0, $t1, $t2, $t3, and $t4.

Instructions for our MIPS is given below:

Instruction ID	Instruction Type	Instruction
A	Arithmetic	add
B	Arithmetic	addi
C	Arithmetic	sub
D	Arithmetic	subi
E	Logic	and
F	Logic	andi
G	Logic	or
H	Logic	ori
I	Logic	sll
J	Logic	srl
K	Logic	nor
L	Memory	lw
M	Memory	sw
N	Control	beq
O	Control	bneq
P	Control	j

Fig: Instruction Set Description

Our MIPS instruction would be 16 bits long following these 4 formats.

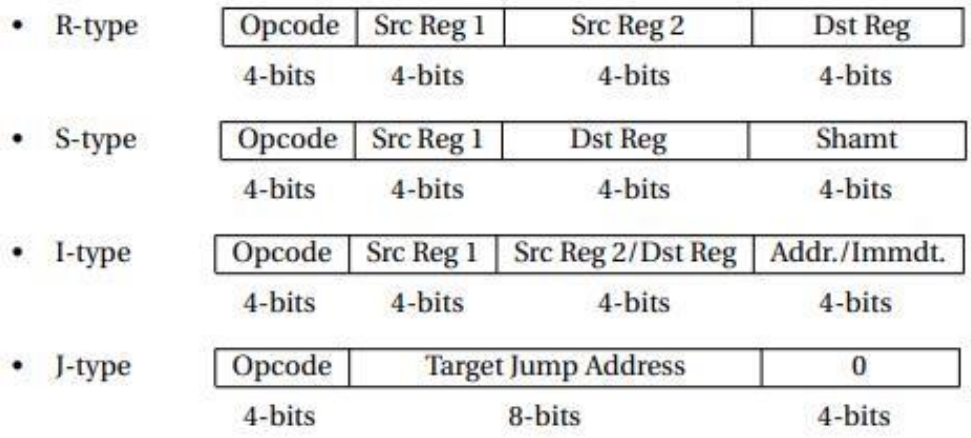


Fig: MIPS Instruction Format

Opcodes of the instructions are of 4 bits, so between 0 and 15. We're given the Instruction assignment GAPOLNDMJKHCFEIB. So, our instruction set would be something like this.

Opcode	Instruction Type	Instruction
0000	Logic	or
0001	Arithmetic	add
0010	Control	j
0011	Control	bneq
0100	Memory	sw
0101	Control	beq
0110	Arithmetic	subi
0111	Memory	lw
1000	Logic	srl
1001	Logic	nor
1010	Logic	ori
1011	Arithmetic	sub
1100	Logic	andi
1101	Logic	and
1110	Logic	sll
1111	Arithmetic	addi

Table: Instructions for each opcode

Circuit Diagram:

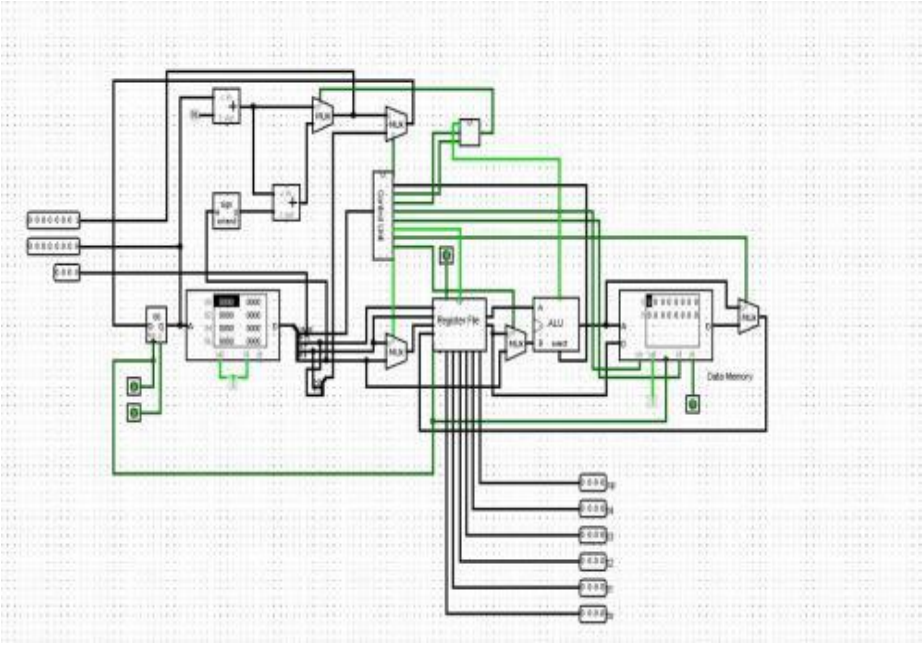


Fig: 4-bit processor in logisim

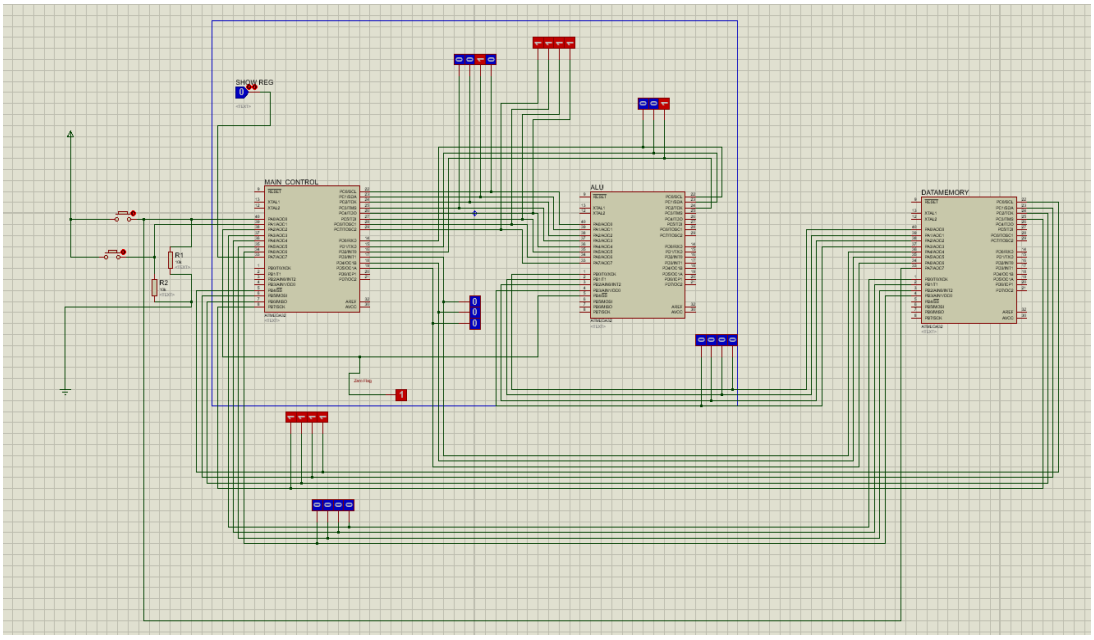


Fig: MIPS using proteus

Design Steps:

In our MIPS architecture, the following components were used to build up the entire structure.

1. Instruction Memory:

We utilized RAM for instruction storage, converting input instructions to 16-bit MIPS codes. These were split into four 4-bit segments: destination register/immediate value, second source register/destination register, first source register, and Opcode. The instruction memory output a new instruction as the PC value incremented.

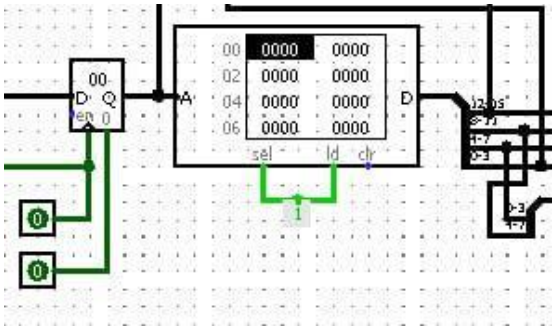


Fig: Instruction Memory

2. ALU:

Our task is to design a 4-bit ALU for arithmetic and logical operations. It takes two inputs and an ALUOP value, producing a 4-bit output and a zero-flag indicating if the output is zero.

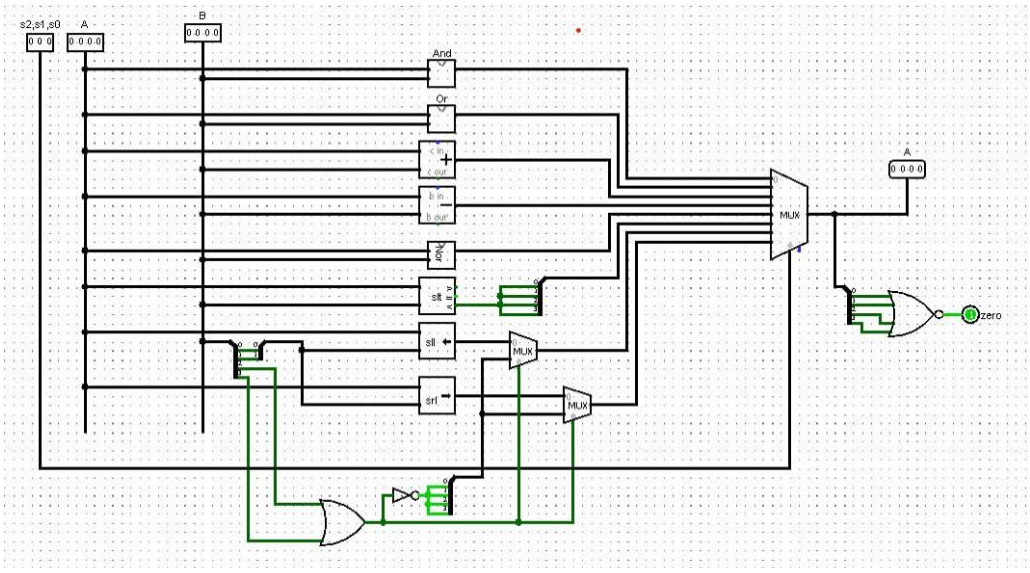


Fig: ALU Main

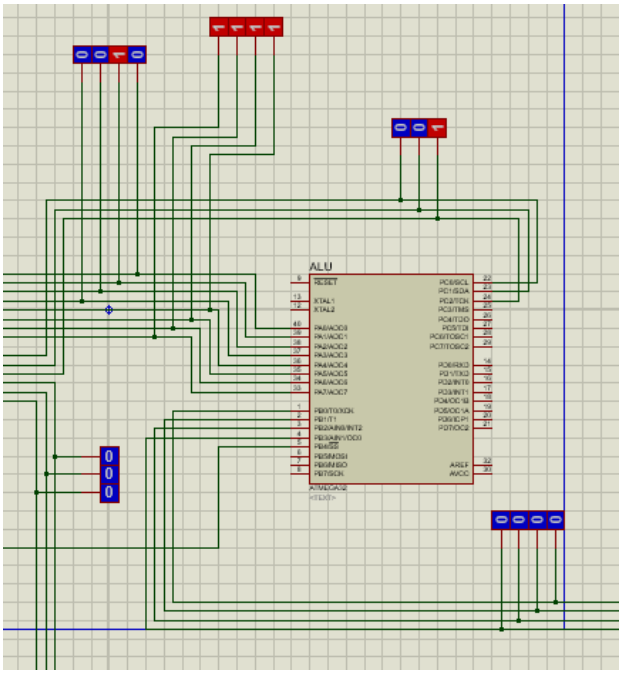


Fig: ALU in proteus

ALU COMPONENTS:

These are the necessary elements used in the main ALU circuit.

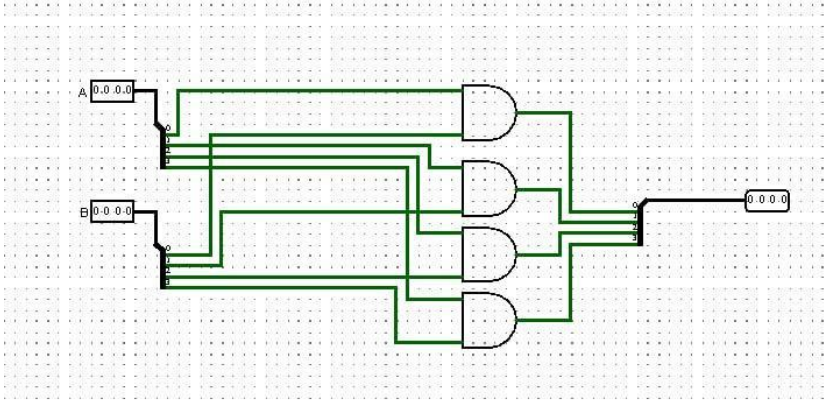


Fig:4-bit AND

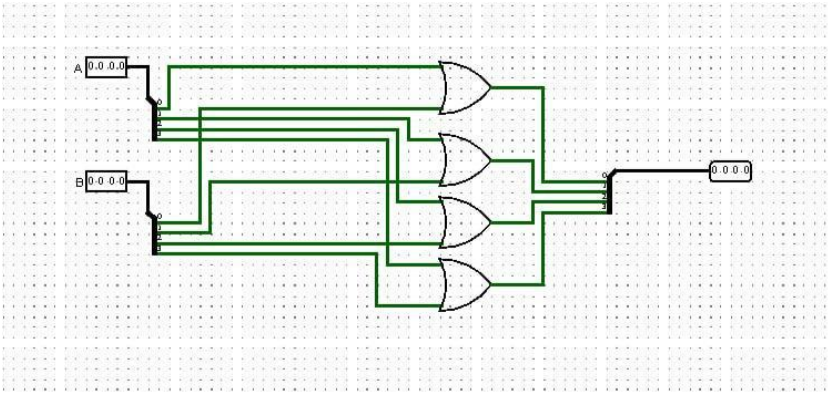


Fig:4-bit OR

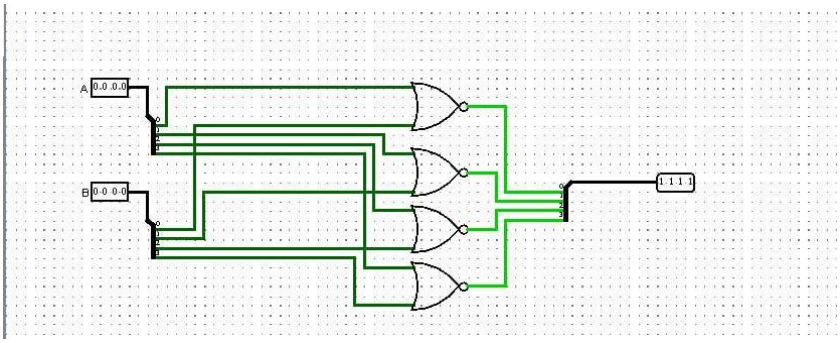


Fig:4-bit NOR

To use the ALU, our circuit takes the opcode along with two 4-bit input values. It also takes beq and bneq flags as inputs from the Branch Circuit.

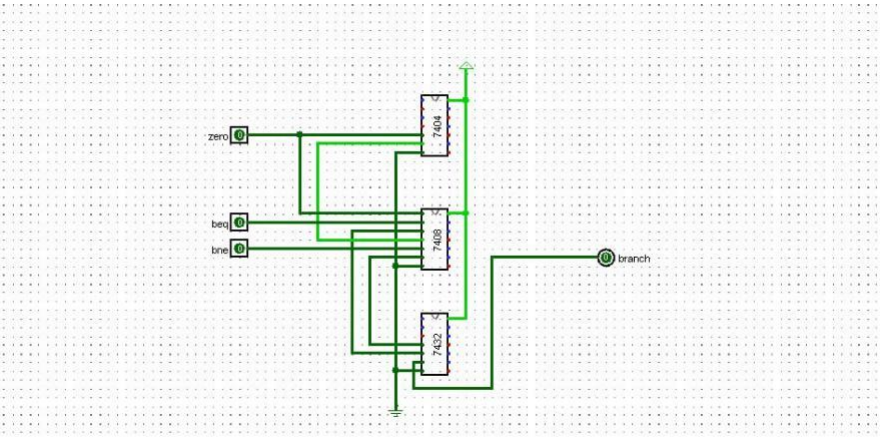


Fig: Branch

3. Control Unit:

In the control unit, we utilize a 16-bit ROM to set MUX operation values according to the provided instruction set for each group. Upon receiving the 4-bit OpCode, we use it to select the corresponding operation from the ROM. The ROM outputs 9 selector bits for the operation and a 3-bit ALUop, which controls the operation of the 4-bit ALU.

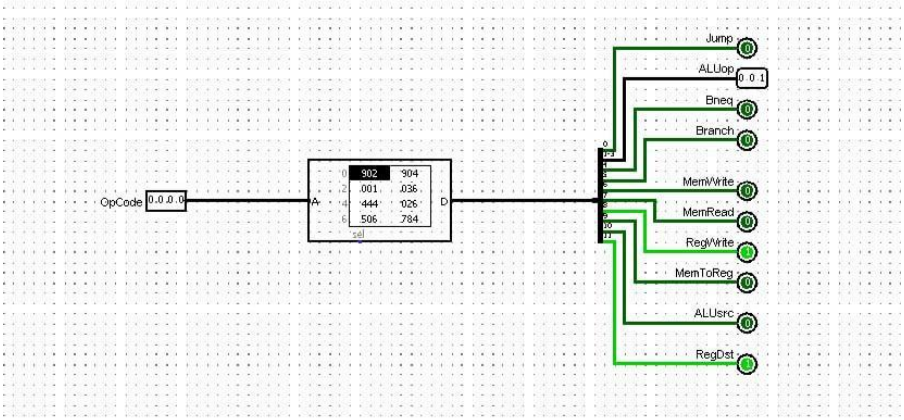


Fig: Control Main

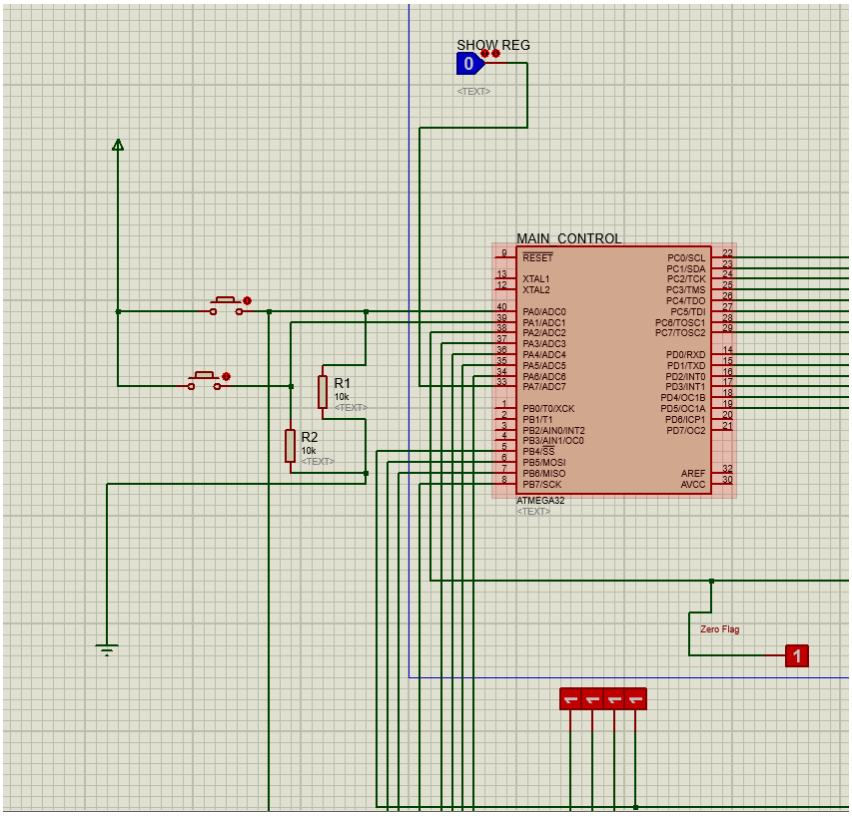


Fig: Control Main in proteus

In proteus, we have kept a ShowReg in Control Main to enable the final result show of the registers.

4. Register File:

We created a register file consisting of 7 registers, including $t0 to $t4 for temporary storage, $sp for stack operations, and $zero for arithmetic with zero. The register file reads from or writes to a register based on the provided register address. When writing, we input the data value and register address, alongside a RegWrite bit to determine if writing is necessary. For store word instructions, writing to the register file is unnecessary.

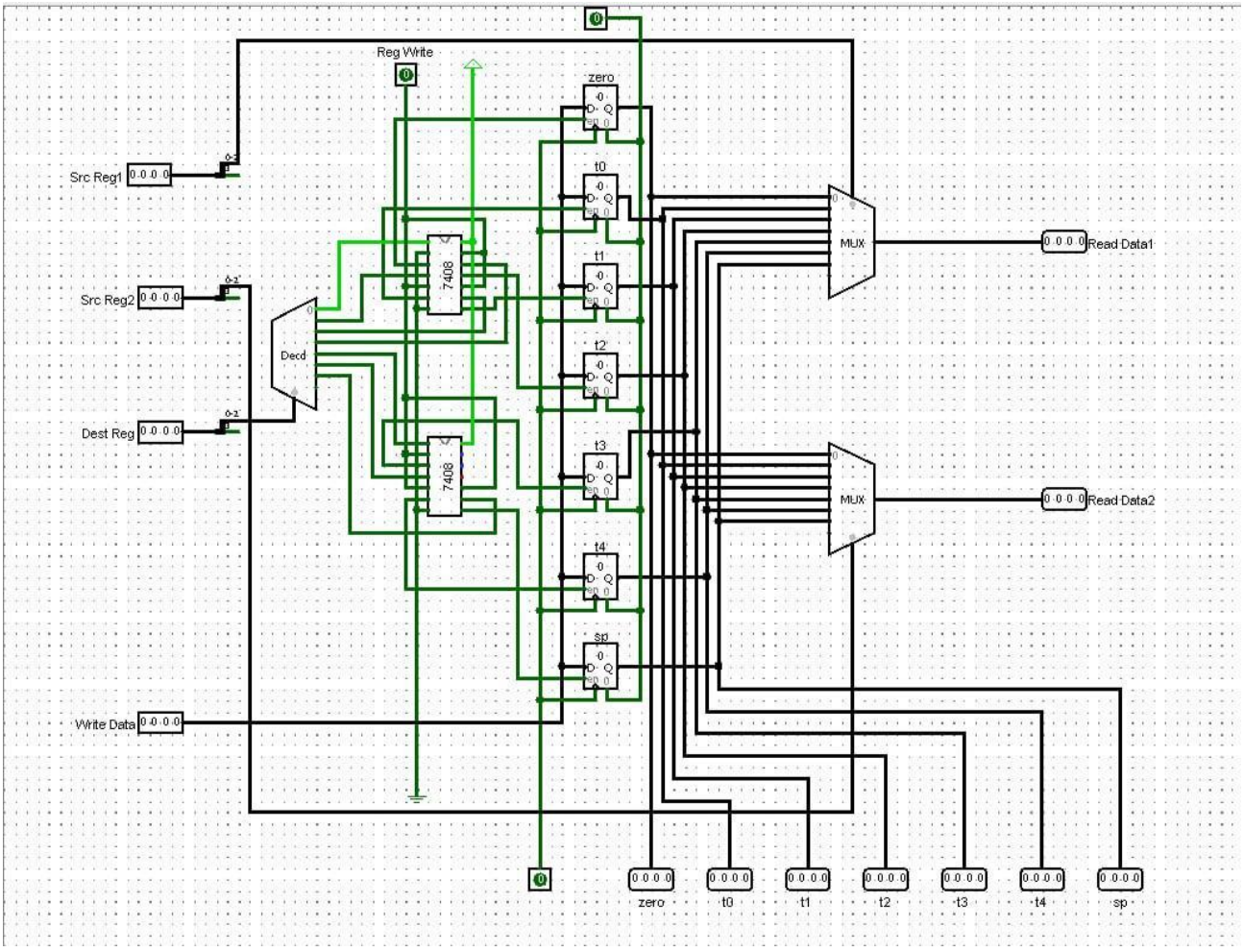


Fig: Register File

5. Data Memory:

Data Memory is our storage for memory. It takes the memory address to write and the data to write in the 16B RAM. Now there can be read from memory in lw instruction and write in memory in store word instruction. So based on two selector bits, we select our operation either to read or write. For memory read (lw), the output is our 4-bit memory data.

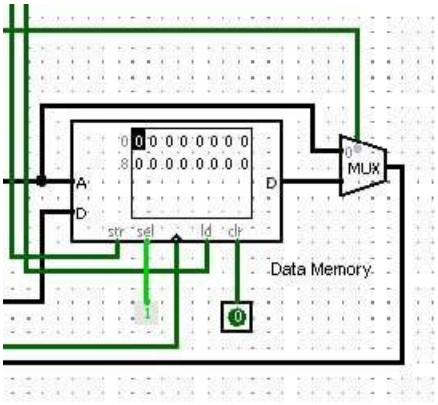


Fig: Data memory

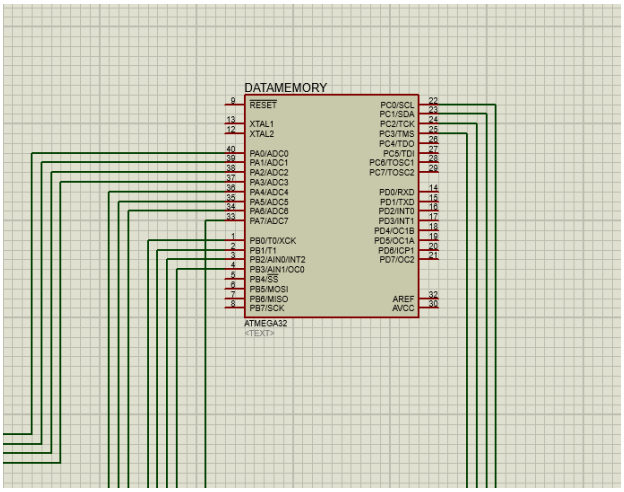


Fig: Data Memory in proteus

Tools and Apparatus:

COMPONENT	AMOUNT
ATmega32	3

Simulator:

Logisim-win-2.7.1

Discussion:

1. At the software level, we employed 7400-library integrated circuits for circuit implementation, while at the hardware level, we executed the necessary coding at the ATMEGA level.

2. In the hardware implementation, we enabled the ShowReg feature in the register file to display the value of each register.

3. Various wire sizes were strategically employed to maintain organization and enhance user comprehension throughout the entire circuit.

4. We meticulously verified the outputs using numerous input test cases to ensure alignment with the expected values from the input file.

5. We integrated PC+1 functionality using separate adder.

Contributions:

1. All necessary elements are bought by 2005111.
2. Simulation parts & codes are done by 2005114.
3. Hardware parts are done by 2005095, 2005096, 2005111, 2005114, 2005119.
4. Report is written by 2005096 & 2005119.